FinTech 512 Midterm Spring 2024: Personal Playlists

Name:_____

NetID: _____

There are five questions, with the point values as shown below. You have 1 hour and 15 minutes. There are 100 points on the exam. Pace yourself accordingly. This exam must be individual work. You may not collaborate with your fellow students. You may use the two sheets of notes you prepared. You may not use any other external resources, e.g., electronic devices. Points will be deducted if you provide more examples than the number listed.

I certify that the work shown on this exam is my own work, and that I have neither given nor received improper assistance of any form in the completion of this work.

Signature: _____

You've been contacted by a local entrepreneur looking for technical help with adapting their IT systems for their music shop. Here is a recent email the entrepreneur has sent you:

> Hello <your name>,
>
> I really appreciate that you have signed up to help with this project and I trust that you find the terms of the contract acceptable.
>
> As we do not have the budget for full-time technical staff, we have to manage our technical infrastructure and services in small, discrete projects. A previous consultant set up our Flask application and the sales database behind it. The application interfaces with our Point-Of-Sale system which handles cash, check, and credit-card sales. We are pleased with many aspects of the system but have a need for an additional feature: personal playlists. Currently, we can sell albums and individual tracks to customers, and we can create playlists that contain lists of tracks. However, we would like to allow customers to create their own playlists to enhance our service to them. To encourage customer participation in the personalized playlist feature, we would like to offer a discount for customers buying tracks that are part of a playlist they have created. In addition, we would like to review weekly, monthly, and annual sales trends to see what the most popular tracks, albums, and genres are, and who the most active buyers are.
>
> We were impressed with your work on the Chinook database queries and we've heard that you have skills in database design and website development. We are looking forward to your technical advice on upgrading our current system.

NetID: _____

Question 1: Requirements

a. (12 pts) Identify three functional requirements (FR) and one non-functional requirement (NFR) in the above email. State the requirement as a sentence and indicate its type (FR, NFR.)

Type	Requirement text
FR	**FR: As a shop owner, I would like to allow customers to create their own playlists to enhance our service to them.** **FR: As a shop owner I would like to be able to offer discounts on customer playlist purchases, to support customer goodwill and word of mouth marketing.**
FR	**FR: As a shop owner, I would like to be able to generate sales trends reports that can be customized by date range (e.g. weekly, monthly, annual) so that I can evaluate what is popular, enabling planning.** **FR: As a staff member, I would like to be able to manage customer playlists using the same system I now use to sell music, so I can apply my experience and so I have less to learn.**
FR	**FR: As a shop owner I would like the system to integrate with my existing Point-Of-Sale system to minimize the complexity of setting up new products and of selling the items we sell.**
NFR	**NFR: The system needs to be usable (Usability) by the shop's current, non-technical, staff.** **NFR: The system needs to be usable (Usability) by the shop's customers.** **NFR: Integration of customer playlists within the existing music shop system (modifiability, maintainability)** **NFR: Precautions against loss and mishandling of the money involved in music sales (Security)**

b. (8 pts) Write two user stories based upon the email.

See FR's written as user stories above.

Question 2: Database Design

We have been informally keeping customer playlist data in a spreadsheet. Here are the first five rows. Determine what tables to add or modify in Chinook to support personal playlists.

Customer	Album Title	Track Name	Playlist Name
Jim Smith	Greatest Hits	Here I Go Again	Music, 90's Music
Jim Smith	Back In Black	For Those About To Rock	Heavy Metal, Music
Jim Smith	Meus	Grito De Alerta	90's Music, Brazilian Music
Alex Rocha	Holst: The Planets	Jupiter, Bringer of Jollity	Classical, Classical 101
Dom King	Nevermind	Come As You Are	Grunge, 90's Music

[Update the chinook schema (last page) to track the 'spreadsheet' data and link it to the invoice. If you need to modify an existing table, describe the changes to be made to those table(s)]

a. (20 pts) Normalize the data in the given data rows to the third normal form. Show the table (relation) names, field names, and the reformatted data.

Example solution:
Use existing customers, albums, tracks, playlists, and playlist_tracks relation names and attributes. Needs relation for customer playlists.

customers(Customerid, FirstName, LastName):
1,Jim,Smith | 2,Alex,Rocha | 3,Dom,King

albums(AlbumId,Title): 1,Greatest Hits | 2,Back In Black | 3,Meus | 4,Holst: The Planets | 5,Nevermind

tracks(TrackId,Name): 1,Here I Go Again | 2,For Those About To Rock | 3,Grito De Alerta | 4,Jupiter, Bringer of Jollity | 5,Come As You Are

playlists(PlaylistId,Name): 1,Music | 2,90's Music | 3,Heavy Metal | 4,Brailian Music | 5,Classical | 6,Classical 101 | 7,Grunge 8,90's Music

playlist_track(PlaylistID,TrackId): 1,1 | 2,1 | 3,2 | 1,2 | 4,3 | 5,4 | 6,4 | 7,5 | 8,5

customer_playlist(CustomerId, PlaylistId): 1,1 | 1,2 | 1,3 | 1,4 | 2,5 | 2,6 | 3,7

Creating a customer_playlist table with customerID, playlistID, and playlistName is also sufficient.

b. (10 pts) Write the SQL DDL statement(s) for the relations you defined in part a.

Something that represents the customer-playlist relationship sketched in part a, e.g.

```
CREATE TABLE Customer_Playlists(
        CustomerId INTEGER,
        PlaylistId INTEGER,
        FOREIGN KEY (CustomerId) REFERENCES Customers(CustomerId),
        FOREIGN KEY (PlaylistId) REFERENCES Playlists(PlaylistId)
        );

-- or –

CREATE TABLE Customer_Playlists(
        CustomerId INTEGER primary key,
        PlaylistId INTEGER primary key,
        Name varchar(100) not null,
        FOREIGN KEY (CustomerId) REFERENCES Customers(CustomerId)
);
```

as long as the playlistid is unique between the playlists table and this table, this solution suffices

Question 3: Testing

The previous contractor developed code for calculating playlist discounts, Python function 'playlist_discount', **shown on the second to the last page of the exam**. Given that both money and customer goodwill are at stake in the proper execution of this code, we would like an assessment of how thoroughly the code has been tested.

Here are the test cases the contractor provided:
The invoice has 3 items, and no playlist items. The playlist has three items.
The invoice has 6 items, three of which are playlist items. The playlist has three items.

a. (5 pts) For the code and the tests given above, what level of coverage (path, condition, statement, none of the above) is achieved?

   None of the above

b. (5 pts) What test cases need to be added to achieve full statement coverage? (You can give just the parameter values, as in the test case examples above)

   test case(s) that cover lines 21, 26, and 27, which are the lines not covered by the two example test cases.

   e.g. line 21 (s/p < .5):
   invoice(6,2), playlist(6) or invoice(6,2), playlist(2)
   lines 26, 27 (s > 2, s/p < .5): invoice(6,2), playlist(2)

c. (5 pts) What test cases need to be added to achieve full condition coverage? (You can give just the parameter values, as in the test case examples above)

   Condition coverage asks that each conditional clause is executed. Need to add test case to force s/n > 0.5 to be executed.

   Branch Coverage (not asked):
   given: invoice(3,0), playlist(3) invoice(6,3), playlist(3)

   Test cases that address:
   - **empty invoice items, e.g. invoice(0,0),playlist(0)**
   - **empty playlist_items, e.g. invoice(1,0),playlist(0)**
   - **invoice with no tracks shared with playlist, e.g. invoice(1,0),playlist(1)**
   - **pd between 0 and 0.10, e.g. invoice(6,2), playlist(6)**

Question 4: Refactoring

a. (9 pts) We've heard that 'Code Smells' are wrong. Name three code smells that the playlist_discount code contains.

Example Code Smells

- **Mysterious Name**
- **Long Function**
- **Comments (and their absence)**
- **Primitive Obsession**
- **Divergent Change**
- **Duplicated Code**

b. (9 pts) List three refactorings that can be applied to improve the code and address the code smells identified in part a. Be specific, e.g., if you apply 'Extract Function', give the function name and its parameters.) Write code, noting the name of the refactoring in a comment.

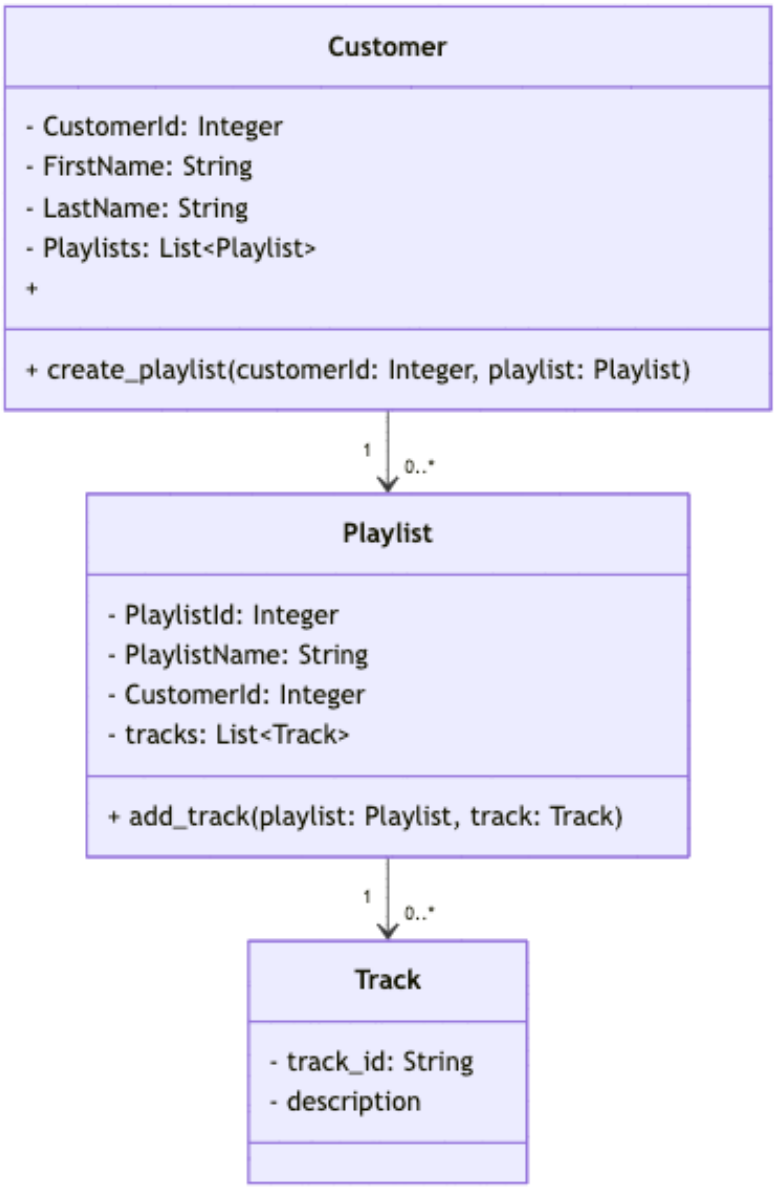
Example Refactorings

- **Rename Variable**
- **Extract Function (Extract Method)**
- **Extract Variable**
- **Change Function Declaration**
- **Replace Primitive with Object**

Question 5: Design

(17 pts) We've heard that object-oriented design is the best approach. Create a UML diagram that covers tracking users and their associated playlists. Ensure you include both properties and behavior. Use the email as the starting point for what to include in this design.

Need to see classes listed with associated state and behavior. Expect to see at least a customer and a playlist object as a minimum.

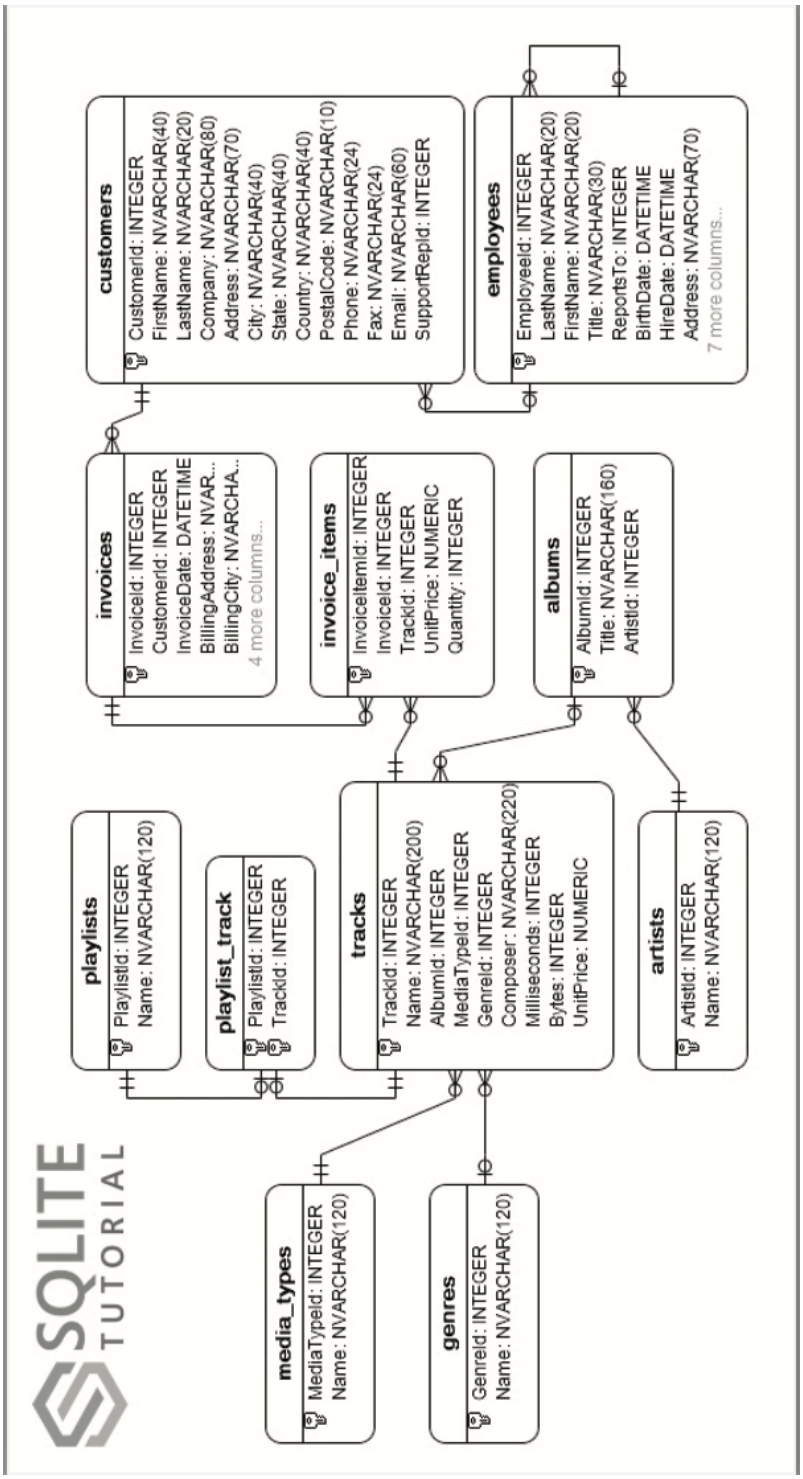


```
def playlist_discount(invoice_id, customer_id, playlist_id):
    invoice_items = get_db().execute("SELECT trackid, unitPrice, quantity "
                                     "  FROM INVOICE_ITEMS "
                                     "  WHERE invoiceId = ?",
                                     (invoice_id,)).fetchall()

    playlist_items = get_db().execute("SELECT trackid FROM playlist_track "
                                      "  WHERE playlistid = ?", (playlist_id))

    n = s = p = pd = 0
    for item in invoice_items:
        n += 1
        for id in playlist_items:
            P = len(playlist_items)
            if item["trackid"] == id:
                s += 1
    d = 0.0
    if p > 0 and s > 0:
        pd = .20 * s/p
        if pd > 0 and pd < .10:
            od = .05
        else:
            od = .1
        if s > 2 or s/n > 0.5:
            d = max(od,pd)
        else:
            d = min(od,pd)
    return d
```

NetID: _____


SQLITE
TUTORIAL
customers
CustomerId: INTEGER
FirstName: NVARCHAR(40)
LastName: NVARCHAR(20)
Company: NVARCHAR(80)
Address: NVARCHAR(70)
City: NVARCHAR(40)
State: NVARCHAR(40)
Country: NVARCHAR(40)
PostalCode: NVARCHAR(10)
Phone: NVARCHAR(24)
Fax: NVARCHAR(24)
Email: NVARCHAR(60)
SupportRepId: INTEGER
employees
EmployeeId: INTEGER
LastName: NVARCHAR(20)
FirstName: NVARCHAR(20)
Title: NVARCHAR(30)
ReportsTo: INTEGER
BirthDate: DATETIME
HireDate: DATETIME
Address: NVARCHAR(70)
7 more columns...
invoices
InvoiceId: INTEGER
CustomerId: INTEGER
InvoiceDate: DATETIME
BillingAddress: NVAR...
BillingCity: NVARCHA...
4 more columns...
invoice_items
InvoiceItemId: INTEGER
InvoiceId: INTEGER
TrackId: INTEGER
UnitPrice: NUMERIC
Quantity: INTEGER
albums
AlbumId: INTEGER
Title: NVARCHAR(160)
ArtistId: INTEGER
playlists
PlaylistId: INTEGER
Name: NVARCHAR(120)
playlist_track
PlaylistId: INTEGER
TrackId: INTEGER
tracks
TrackId: INTEGER
Name: NVARCHAR(200)
AlbumId: INTEGER
MediaTypeId: INTEGER
GenreId: INTEGER
Composer: NVARCHAR(220)
Milliseconds: INTEGER
Bytes: INTEGER
UnitPrice: NUMERIC
artists
ArtistId: INTEGER
Name: NVARCHAR(120)
media_types
MediaTypeId: INTEGER
Name: NVARCHAR(120)
genres
GenreId: INTEGER
Name: NVARCHAR(120)